KỊCH BẢN BÁO CÁO GIỮA KỲ — NHÓM 11

Đề tài: Phân hệ đóng học phí trên iBanking theo kiến trúc Microservices

Thời lượng: 10 phút trình bày và demo, 5 phút hỏi đáp

Thành viên: Lê Nam Giang (523H0020), Nguyễn Gia Nhật Khánh (523H0149), Nguyễn Hoàng Thanh Minh (519H0194)

Phần	Thời gian	Mốc đồng hồ	Người nói (gợi ý)	Màn hình
1. Tổng quan	1 phút	0:00 – 1:00	Thành viên A	Slide 1
2. Phân tích và thiết kế	2 phút	1:00 – 3:00	Thành viên A	Slide 2, 3, 4
3. Giải pháp kỹ thuật	2 phút	3:00 – 5:00	Thành viên B	Slide 5, 6, 7
4. Demo	5 phút	5:00 – 10:00	Thành viên C	Trình duyệt và terminal
5. Hỏi đáp	5 phút	10:00 – 15:00	Cả nhóm	Mã nguồn

Phân vai chỉ là gợi ý, nhóm tự điền tên. Người demo nên là người chia sẻ màn hình suốt buổi để không mất thời gian đổi người chia sẻ.

A. Chuẩn bị trước giờ báo cáo

Tối hôm trước

1. Chạy `./setup.sh` và chờ dòng "Hệ thống: UP". Script cũng tự tạo 9 giao dịch mẫu cho 6 tài khoản phụ, không đụng tới tài khoản và MSSV dùng trong demo.
2. Kiểm tra `522H0001` còn **chưa thanh toán** (đăng nhập `minhnguyen`, tra cứu thử, không bấm xác nhận). Nếu đã đóng trong lần tập trước thì nạp lại dữ liệu gốc: `docker compose down -v` rồi `./setup.sh`. Lệnh này xoá toàn bộ dữ liệu cũ.
3. Tập trọn bài một lần có bấm giờ, sau đó nạp lại dữ liệu gốc như bước 2.

30 phút trước lượt báo cáo

1. Mở Docker Desktop, chạy `./setup.sh`.
2. Mở sẵn 5 tab trình duyệt theo đúng thứ tự:
   - Slide: mở `docs/Slide.pptx` bằng PowerPoint
   - http://localhost:3000 (đã đăng nhập `minhnguyen` / `123456`)
   - http://localhost:8025 (MailHog, hộp thư OTP)
   - http://localhost:16686 (Jaeger)
   - http://localhost:8004/docs (Swagger của payment-service)
3. Mở terminal tại thư mục dự án, phóng to cỡ chữ, gõ sẵn nhưng chưa Enter:
   `.venv/bin/python -m pytest tests/test_03_concurrency.py -v`
4. Mở VS Code tại thư mục dự án, mở sẵn 4 file ở mục D.
5. Tắt thông báo, cắm sạc, mở webcam, thử chia sẻ màn hình.

Thứ tự demo bắt buộc: làm phần giao diện trước, chạy pytest sau. Bộ test nạp tiền và tiêu tiền trên chính ba tài khoản mẫu, riêng `quocbao` sẽ còn 0 đồng sau khi chạy.

B. Lời thoại phần trình bày (5 phút)

Slide 1 — Tổng quan (1 phút)

Trên slide: tên đề tài, tên nhóm, 4 chức năng chính, dòng công nghệ.

> Kính chào thầy, nhóm 11 xin trình bày đồ án giữa kỳ: phân hệ đóng học phí trên ứng dụng iBanking, xây dựng theo kiến trúc microservices.
>
> Nghiệp vụ như sau: người dùng iBanking đăng nhập, nhập mã số sinh viên để tra cứu học phí, rồi thanh toán cho chính mình hoặc cho sinh viên khác. Giao dịch chỉ được thực hiện khi số dư đủ, người dùng đồng ý điều khoản và nhập đúng mã OTP gửi qua email. Sau khi thành công, hệ thống trừ tiền, gạch nợ học phí, lưu lịch sử và gửi email biên lai.
>
> Hệ thống gồm sáu service nghiệp vụ, một API Gateway và một giao diện web, viết bằng Python FastAPI, PostgreSQL và chạy bằng Docker Compose với 11 container.

Slide 2 — Use Case (30 giây)

Trên slide: hình `docs/report-assets/figure-01-*.png`.

> Hệ thống có một tác nhân chính là người dùng iBanking với bảy use case: đăng nhập, xem số dư, tra cứu học phí, khởi tạo giao dịch, xác thực OTP, gửi lại OTP và xem lịch sử. Hai tác nhân phụ là hệ thống email và hệ thống đào tạo. Đề bài không yêu cầu đăng ký tài khoản nên nhóm không làm chức năng này.

Slide 3 — ERD (40 giây)

Trên slide: hình `figure-03-*.png` và bảng 6 database.

> Về dữ liệu, nhóm áp dụng database per service: sáu service sở hữu sáu database riêng với tổng cộng chín bảng. Quan hệ trong cùng một database là khóa ngoại thật, còn quan hệ xuyên service chỉ là tham chiếu logic qua ID, không có khóa ngoại vật lý.

> Các ràng buộc quan trọng đặt ngay ở tầng database: số dư không âm, mã tham chiếu bút toán là duy nhất, và mỗi khoản học phí chỉ có tối đa một giao dịch thành công.

Slide 4 — Kiến trúc Microservices (50 giây)

Trên slide: hình `figure-04-*.png`.

> Web chỉ gọi một địa chỉ duy nhất là API Gateway. Gateway xác thực JWT, định tuyến và chặn các đường dẫn nội bộ.
>
> Phía sau là sáu service, mỗi service một trách nhiệm: auth quản lý người dùng và phát hành JWT; account giữ số dư và sổ cái; tuition giữ sinh viên và công nợ; otp sinh và xác thực mã; notification là cổng gửi email duy nhất; và payment là bộ điều phối, không tự sửa số dư hay học phí mà gọi các service sở hữu dữ liệu.
>
> Nhóm phân rã theo năng lực nghiệp vụ, nên đổi chính sách OTP chỉ đụng otp-service, đổi nhà cung cấp email chỉ đụng notification-service.

Slide 5 — Giao tiếp và OTP (40 giây)

Trên slide: hình sequence `figure-05-*.png`, 5 gạch đầu dòng về OTP.

> Các service giao tiếp bằng REST JSON đồng bộ, vì người dùng đang chờ kết quả giao dịch. Lời gọi nội bộ đi qua đường dẫn internal, có token riêng và bị gateway chặn từ bên ngoài. Mọi request được truy vết bằng OpenTelemetry và xem trên Jaeger.
>
> Mã OTP gồm sáu chữ số, gắn với đúng một giao dịch, hết hạn sau năm phút, chỉ dùng một lần, bị khóa sau năm lần nhập sai, và chỉ lưu dạng băm SHA-256 chứ không lưu mã gốc. Khi gửi lại mã thì mã cũ mất hiệu lực ngay.

Slide 6 — Transaction: Saga có bù trừ (40 giây)

Trên slide: hình `figure-07-*.png`.

> Vì mỗi service có database riêng nên không thể dùng một transaction chung. Nhóm chia thành hai tầng. Trong một service, dùng transaction ACID của PostgreSQL. Xuyên service, dùng Saga do payment-service điều phối: xác thực OTP, trừ tiền, gạch nợ. Nếu gạch nợ thất bại thì chạy bước bù trừ là hoàn tiền.
>
> Mỗi bước đều idempotent theo mã giao dịch nên gọi lại không trừ tiền hai lần. Nếu service sập giữa chừng, một job đối soát chạy mỗi 30 giây sẽ đọc lại sổ cái và công nợ để hoàn tất hoặc hoàn tiền.

Slide 7 — Concurrency: hai tình huống đề bài (40 giây)

Trên slide: hình `figure-08-*.png` và `figure-09-*.png` đặt cạnh nhau.

> Tình huống một, nhiều giao dịch trên cùng một tài khoản: account-service khóa dòng tài khoản bằng SELECT FOR UPDATE. Giao dịch thứ hai phải chờ, đọc số dư mới và bị từ chối vì không đủ tiền. Số dư không bao giờ âm.
>
> Tình huống hai, nhiều tài khoản cùng đóng một khoản học phí: tuition-service khóa dòng học phí. Chỉ một người gạch nợ thành công, người còn lại được hoàn tiền tự động.
>
> Ngoài khóa dòng còn có lớp dự phòng là các ràng buộc CHECK và UNIQUE. Sau đây nhóm xin demo trực tiếp.

Chuyển sang slide 8 ("Demo hệ thống") rồi đổi sang trình duyệt. Slide nằm ở `docs/Slide.pptx`; lời thoại trên đã có sẵn trong ghi chú diễn giả của từng slide.

C. Kịch bản demo (5 phút)

Mốc	Thao tác	Câu nói kèm theo
5:00	Tab web, đã đăng nhập `minhnguyen`. Chỉ vào khối người nộp tiền.	"Thông tin người nộp tự điền từ tài khoản và không sửa được. Số dư hiện tại 50 triệu."
5:20	Nhập `522H0004`, bấm Tra cứu.	"Khoản đã đóng thì hệ thống chặn ngay từ bước tra cứu."
5:35	Nhập `522H0001`, Tra cứu. Tick điều khoản. Bấm Xác nhận giao dịch.	"Học phí 9,5 triệu. Nút xác nhận chỉ bật khi đủ ba điều kiện: có công nợ, đủ số dư, đồng ý điều khoản."
6:00	Hộp thoại OTP hiện. Nhập sai `000000`.	"Nhập sai thì bị từ chối và báo số lần thử còn lại."
6:15	Sang tab MailHog, mở email, đọc mã. Quay lại nhập đúng.	"OTP gửi qua email, có đồng hồ đếm ngược 5 phút."

Mốc	Thao tác	Câu nói kèm theo
6:45	Hộp thoại thành công. Mở "Xem các bước xử lý".	"Số dư còn 40,5 triệu. Đây là nhật ký saga: xác thực OTP, trừ tiền, gạch nợ, gửi biên lai."
7:05	Tab MailHog: email biên lai. Tab web: Lịch sử giao dịch.	"Biên lai đã về email, lịch sử ghi nhận giao dịch và biến động số dư."
7:25	Terminal, Enter lệnh pytest đã gõ sẵn.	"Hai tình huống đồng thời cần nhiều request cùng một thời điểm nên nhóm dùng test tự động, gọi vào đúng hệ thống đang chạy."
7:35	Trong lúc test chạy, nói:	"Test 5.1: một tài khoản, hai giao dịch, số dư chỉ đủ một. Test 5.2: hai người cùng đóng một khoản học phí. Test thứ ba: bấm xác nhận ba lần cùng lúc."
8:00	Chỉ 3 dòng PASSED.	"Kết quả: 5.1 có một thành công, một bị từ chối vì thiếu số dư, số dư cuối bằng 0. 5.2 có một thành công, một thất bại và được hoàn tiền."
8:20	Tab web, F5, Lịch sử giao dịch. Tìm giao dịch thất bại mới nhất, bấm Chi tiết. Nếu `minhnguyen` không có thì đăng nhập `thuha`.	"Đây là giao dịch thua trong tình huống 5.2: trừ tiền OK, gạch nợ thất bại, hoàn tiền bù trừ. Bảng biến động số dư có một dòng trừ và một dòng cộng 1 triệu."
9:00	Tab Jaeger: chọn service `api-gateway`, Find Traces, mở trace POST payments.	"Một request đi qua gateway, payment, tuition, account, otp và notification với cùng một trace ID."
9:30	Terminal: `docker compose ps`	"11 container: sáu service, gateway, PostgreSQL, MailHog, web và Jaeger. Phần demo của nhóm đến đây là hết, xin mời thầy đặt câu hỏi."

Nếu bị trễ giờ: bỏ bước 5:20 và bước Jaeger 9:00. Không được bỏ luồng thành công và pytest.

Nếu thầy yêu cầu làm tay tình huống đồng thời (chỉ làm được khi chưa chạy pytest hoặc số dư còn đủ):

- *Tình huống 5.2:* cửa sổ thường đăng nhập `minhnguyen`, cửa sổ ẩn danh đăng nhập `thuha`. Cả hai tra cứu `522H0005` (11 triệu), cùng bấm Xác nhận giao dịch, lấy hai mã OTP trong MailHog, nhập sẵn rồi bấm Xác nhận ở hai cửa sổ liên tiếp nhau. Một bên thành công, bên kia báo học phí đã được thanh toán và số dư không đổi.
- *Tình huống 5.1:* đăng nhập `quocbao` (15 triệu) ở hai tab. Tab một tạo giao dịch cho `522H0002` (8,75 triệu), tab hai cho `522H0003` (10,2 triệu). Cả hai đều qua bước kiểm tra ban đầu vì từng khoản nhỏ hơn số dư. Nhập OTP và xác nhận cả hai: một thành công, một báo số dư không đủ.

Nếu có sự cố

Sự cố	Xử lý
Web không mở được	`docker compose ps`, rồi `docker compose up -d --wait`
Không thấy email OTP	F5 tab MailHog; bấm "Gửi lại mã" trên web
`522H0001` đã đóng rồi	Dùng `522H0002` hoặc `522H0003` thay thế
pytest lỗi kết nối	Kiểm tra http://localhost:8080/health/services
Mất mạng Google Meet	Hệ thống chạy cục bộ, không cần Internet; chỉ cần vào lại phòng họp

D. Hỏi đáp: câu trả lời ngắn và nơi mở mã nguồn

Mở sẵn 4 file sau trong VS Code:

Nội dung thầy có thể yêu cầu xem	**File và dòng**
Saga: OTP → trừ tiền → gạch nợ → hoàn tiền	`services/payment-service/app/main.py`, hàm `confirm_payment` dòng 217; trừ tiền 252; gạch nợ 271; hoàn tiền 283
Khóa dòng khi trừ tiền (tình huống 5.1)	`services/account-service/app/main.py` dòng 49
Khóa dòng khi gạch nợ (tình huống 5.2)	`services/tuition-service/app/main.py` dòng 46
Sinh và xác thực OTP	`services/otp-service/app/main.py`, `issue` dòng 67, `verify` dòng 136
Job đối soát giao dịch treo	`services/payment-service/app/reconciliation.py` dòng 98; bảng quyết định ở `decisions.py` dòng 12
Gateway: JWT, định tuyến, chặn internal	`services/api-gateway/app/main.py`
Test đồng thời	`tests/test_03_concurrency.py`

1. Vì sao không dùng một transaction cho cả hệ thống? Mỗi service có database riêng nên không có transaction ACID xuyên service. Nhóm không dùng two-phase commit vì nó khóa tài nguyên lâu và phụ thuộc bộ điều phối. Nhóm dùng Saga: mỗi bước là một transaction cục bộ, bước sau hỏng thì bù trừ bước trước.

2. Vì sao chọn Saga orchestration mà không phải choreography? Giao dịch tài chính cần một nơi duy nhất biết giao dịch đang ở bước nào và vì sao hỏng. payment-service giữ trạng thái và ghi bảng `payment_events` cho từng bước, dễ kiểm toán và dễ bù trừ.

3. Vì sao trừ tiền trước rồi mới gạch nợ? Trừ tiền có thể hoàn lại an toàn bằng một bút toán ghi có. Nếu gạch nợ trước mà trừ tiền thất bại, sẽ có lúc học phí hiện "đã đóng" trong khi chưa ai trả tiền.

4. Nếu hệ thống sập ngay sau khi trừ tiền thì sao? Giao dịch nằm ở trạng thái PROCESSING. Job đối soát chạy mỗi 30 giây, xét các giao dịch treo quá 60 giây, đọc lại sổ cái và công nợ. Đã gạch nợ và có bút toán trừ tiền thì hoàn tất; chỉ mới trừ tiền thì hoàn tiền; đã gạch nợ mà không có tiền thì hủy gạch nợ.

5. Vì sao dùng khóa bi quan (SELECT FOR UPDATE) mà không dùng khóa lạc quan? Thao tác tiền cần kết quả dứt khoát ngay, không muốn người dùng phải thử lại. Khóa dòng chỉ giữ trong một transaction rất ngắn và chỉ khóa đúng một dòng tài khoản hoặc một dòng học phí, nên các tài khoản khác không bị ảnh hưởng.

6. Mức cô lập là gì? Có bị deadlock không? READ COMMITTED, mặc định của PostgreSQL, vì mọi điểm tranh chấp đã được khóa tường minh. Mỗi transaction trong account và tuition chỉ khóa một dòng nên không có vòng chờ khóa giữa hai transaction.

7. Idempotency làm như thế nào? Ba lớp. Tạo giao dịch dùng header Idempotency-Key, bấm hai lần trả về giao dịch cũ. Trừ tiền dùng reference bằng mã giao dịch, có ràng buộc UNIQUE. Hoàn tiền dùng reference là mã giao dịch kèm hậu tố refund. Gọi lại cùng reference thì trả kết quả cũ, không ghi sổ lần hai.

8. OTP có an toàn không? Hai giao dịch có thể trùng mã không? Mã lưu dạng băm SHA-256 có pepper, gắn với mã giao dịch, hạn 5 phút, dùng một lần, khóa sau 5 lần sai. Có partial unique index trên mã băm của các OTP còn hiệu lực nên hai giao dịch đang chờ không thể trùng mã.

9. Đây có phải microservices thật không, hay monolith chia thư mục? Mỗi service là một container, một tiến trình, một database riêng và chỉ giao tiếp qua REST. Có thể chứng minh bằng cách tắt một service: `docker compose stop tuition-service`, tra cứu học phí sẽ trả 503 trong khi đăng nhập và xem số dư vẫn chạy. Sau đó `docker compose start tuition-service`.

10. Vì sao chỉ có một PostgreSQL? Để tiết kiệm tài nguyên máy demo. Đó là một instance chứa sáu database tách biệt, mỗi service chỉ biết chuỗi kết nối của mình. Tách thành sáu instance chỉ cần đổi biến `DATABASE_URL`, không sửa mã.

11. Vì sao REST đồng bộ mà không dùng message broker? Người dùng đang chờ kết quả nên cần phản hồi ngay. Phần nên bất đồng bộ là gửi email đã được tách riêng: gửi biên lai lỗi không làm hỏng giao dịch đã thành công. Hướng phát triển là đưa bước này sang Kafka hoặc RabbitMQ kèm outbox.

12. Bảo mật giữa các service? JWT ký HS256, gateway kiểm tra trước và từng service kiểm tra lại. Đường dẫn internal yêu cầu header X-Internal-Token và bị gateway chặn. Mật khẩu băm bcrypt.

13. Hệ thống còn hạn chế gì? Nhất quán cuối cùng chứ không tức thời khi có sự cố. payment-service giữ một kết nối database trong lúc gọi HTTP, chưa phù hợp tải lớn. Jaeger lưu trace trong bộ nhớ. Email biên lai chưa có cơ chế gửi lại tự động. Dữ liệu mẫu gồm 10 tài khoản iBanking, 30 sinh viên thuộc 7 khoa và 9 giao dịch mẫu được tạo bằng chính luồng thanh toán thật.

14. Kiểm thử ra sao? 89 ca: 42 unit test không cần Docker và 47 ca tích hợp chạy trên hệ thống thật với PostgreSQL thật. Kết quả 88 đạt và 1 ca OTP hết hạn chạy riêng với thời hạn rút ngắn để không phải chờ 5 phút.

E. Bảng tra nhanh

Tài khoản (mật khẩu chung `123456`)

Username	Số dư gốc	Dùng cho
`minhnguyen`	50.000.000	Luồng chính
`thuha`	20.000.000	Người thứ hai trong tình huống 5.2
`quocbao`	15.000.000	Tình huống 5.1 làm tay

Username	Số dư gốc	Dùng cho
`itdu`	1.000.000	Số dư không đủ, nút xác nhận bị khóa
`ngoclan`, `ducthinh`, `hoangyen`, `giahan`, `tuananh`, `camtu`	12 – 40 triệu	Đã có lịch sử giao dịch mẫu; đăng nhập `giahan` để xem 3 giao dịch

Học phí HK1/2026-2027

MSSV	Số tiền	Trạng thái gốc	Dùng cho
`522H0001`	9.500.000	Chưa đóng	Luồng thành công
`522H0002`	8.750.000	Chưa đóng	Dự phòng, 5.1 làm tay
`522H0003`	10.200.000	Chưa đóng	Dự phòng, 5.1 làm tay
`522H0004`	7.300.000	**Đã đóng**	Chặn khoản đã đóng
`522H0005`	11.000.000	Chưa đóng	5.2 làm tay

Ngoài 5 dòng trên còn 25 sinh viên mẫu `522H0006` – `522L0030` thuộc 7 khoa. Sau khi `./setup.sh` tạo xong 9 giao dịch mẫu thì còn 7 khoản chưa đóng: `522H0010`, `522H0011`, `522K0016`, `522B0019`, `522E0022`, `522F0026`, `522L0030`. Dùng khi thầy muốn tra cứu thêm, ví dụ `522H0010` (9.500.000, chưa đóng) hoặc `522H0007` (đã đóng).

Địa chỉ

Thành phần	Địa chỉ
Giao diện web	http://localhost:3000
Hộp thư OTP (MailHog)	http://localhost:8025
API Gateway	http://localhost:8080/docs
Swagger payment-service	http://localhost:8004/docs
Jaeger	http://localhost:16686

Lệnh

Mục đích	Lệnh
Khởi chạy	`./setup.sh`
Test đồng thời	`.venv/bin/python -m pytest tests/test_03_concurrency.py -v`
Toàn bộ test	`.venv/bin/python -m pytest -q`
Trạng thái container	`docker compose ps`
Sức khỏe các service	`curl -s http://localhost:8080/health/services`
Nạp lại dữ liệu gốc (xoá hết dữ liệu)	`docker compose down -v` rồi `./setup.sh`

Con số cần nhớ: 6 service + gateway + web; 6 database, 9 bảng; 11 container; OTP 6 số, 5 phút, 5 lần sai; job đối soát 30 giây, ngưỡng treo 60 giây; 89 ca kiểm thử.